

GroupBy `.apply()` — Deep Dive Notes (Lecture 91 ka mushkil part, detail mein)

Ye file specially isliye bani hai kyunki `apply()` samajhna mushkil lagta hai. Yaha ek chhote example se **step-by-step** dikhaya gaya hai ki `apply()` andar se kya karta hai, phir lecture ke real examples usi lens se explain kiye hain.

PART A: Pehle Basics Yaad Karo (1 line mein)

`groupby('col')` — data ko groups mein **todta** hai (split). `.apply(function)` — har group par tumhara function **chalata** hai (apply). Pandas phir sab results ko wapas **jodta** hai ek final Series/DataFrame mein (combine).

Isko **Split → Apply → Combine** kehte hain. Confusion yahi hota hai ki "Apply" step mein exactly kya ho raha hai aur "Combine" kaise decide hota hai — wahi is file ka focus hai.

PART B: Chhota Toy Example — Poori Machinery Dikhate Hain

Ek chhota fake dataset lete hain taaki dekh sako exactly kya ho raha hai:

```
python

import pandas as pd

toy = pd.DataFrame({
    'Title': ['Avatar', 'Alien', 'Ben Hur', 'Amelie', 'Boyhood'],
    'Genre': ['Action', 'Action', 'Action', 'Comedy', 'Comedy'],
    'Rating': [7.8, 8.4, 8.1, 8.3, 7.9]
})

print(toy)
```

	Title	Genre	Rating
0	Avatar	Action	7.8
1	Alien	Action	8.4
2	Ben Hur	Action	8.1
3	Amelie	Comedy	8.3
4	Boyhood	Comedy	7.9

Step 1 — SPLIT: `groupby('Genre')` andar se kya karta hai

Jaise hi tum `toy.groupby('Genre')` likhte ho, Pandas **poore DataFrame ko chhote-chhote mini-DataFrames mein tod deta hai** — ek mini-DataFrame har unique Genre ke liye:

```
python
```

```
for name, group in toy.groupby('Genre'):
    print(f"Group naam: {name}")
    print(group)
    print()
```

```
Group naam: Action
   Title  Genre  Rating
0  Avatar  Action    7.8
1   Alien  Action    8.4
2  Ben Hur  Action    8.1
```

```
Group naam: Comedy
   Title  Genre  Rating
3  Amelie  Comedy    8.3
4  Boyhood  Comedy    7.9
```

🔑 **Ye sabse zaroori step hai samajhna:** `groupby('Genre')` ek single object nahi, balki **conceptually 2 alag chhote DataFrames** bana deta hai — ek "Action" ka (3 rows), ek "Comedy" ka (2 rows). `.apply()` inhi mini-DataFrames ke saath kaam karega.

Step 2 — APPLY: function har GROUP ke liye ALAG SE chalta hai

Ab maan lo tumhe har genre mein "A" se start hone wali kitni movies hain, wo count karna hai:

```
python

def count_A(x):
    print("--- Function ko ye x mila (ek group ka poora mini-DataFrame): ---")
    print(x)
    result = x['Title'].str.startswith('A').sum()
    print(f"--- Function ne return kiya: {result} ---\n")
    return result

res1 = toy.groupby('Genre').apply(count_A, include_groups=False)
```

Terminal mein exactly ye print hoga (function 2 baar chala — ek baar har group ke liye):

```
--- Function ko ye x mila (ek group ka poora mini-DataFrame): ---
```

	Title	Rating
0	Avatar	7.8
1	Alien	8.4
2	Ben Hur	8.1

```
--- Function ne return kiya: 2 ---
```

```
--- Function ko ye x mila (ek group ka poora mini-DataFrame): ---
```

	Title	Rating
3	Amelie	8.3
4	Boyhood	7.9

```
--- Function ne return kiya: 1 ---
```

🔑 YAHİ ASLİ CONCEPT HAI:

- `apply(count_A)` likhne ka matlab: "har group ke liye `count_A` function ko ek baar chalo, aur us group ka poora mini-DataFrame `x` naam se function ke andar bhej do."
- Function ke andar `x` matlab "abhi jis group pe kaam ho raha hai, uska poora DataFrame" — poore `toy` DataFrame se nahi, sirf **usi ek group ke rows** se.
- Function `x` ke saath kuch bhi kar sakta hai — filter, count, math, naya column banana — **jo bhi normal DataFrame operations tum jaante ho, sab chalega**, kyunki `x` khud ek chhota DataFrame hi hai.

Step 3 — COMBINE: Pandas result ko kaise jodta hai — YE DEPEND KARTA HAI FUNCTION KYA RETURN KARTA HAI

```
python  
  
print(res1)
```

```
Genre  
Action      2  
Comedy      1  
dtype: int64
```

🔑 Function ne har group ke liye **ek single number (scalar)** return kiya tha (2, phir 1). Jab function scalar return karta hai, Pandas un sab scalars ko jod ke ek **Series** bana deta hai, jiska index group ke naam (Genre) hote hain.

Ye "Aggregation-style" apply hai — result poore dataset se CHHOTA ho jaata hai (5 rows se 2 rows — ek row per group).

Ab DOOSRA case dekho — jab function ek POORA DataFrame return karta hai

```
python
```

```
def add_rank(x):
    x = x.copy()
    x['rank_in_genre'] = x['Rating'].rank(ascending=False)
    return x      # 🗝️ poora x hi wapas kar diya, bas ek naya column jod ke

res2 = toy.groupby('Genre').apply(add_rank, include_groups=False)
print(res2)
```

	Title	Rating	rank_in_genre
Genre			
Action 0	Avatar	7.8	3.0
1	Alien	8.4	1.0
2	Ben Hur	8.1	2.0
Comedy 3	Amelie	8.3	1.0
4	Boyhood	7.9	2.0

🗝️ Is baar function ne **poore x ko (naye column ke saath) wapas kiya** — matlab output shape input jaisa hi hai (usi group ke saare rows). Jab function poora DataFrame/Series return karta hai (input jaisa hi shape), Pandas sab groups ke results ko **wapas stack (glue)** kar deta hai — jaise `vstack` — aur ek **extra index level (Genre)** add kar deta hai upar se.

Ye "Transform-style" apply hai — result poore dataset ke **barabar size** ka hi rehta hai, bas naya column/info add ho gayi.

PART C: Do Tarah Ke `apply()` — Yaad Rakhne Ka Tareeka

	Scalar return karo	DataFrame/Series (same shape) return karo
Function kya karta hai	Group ko ek number/value mein "summarize" karta hai	Group ke har row ko transform/modify karta hai
Result kaisa aata hai	Series — index = group names, ek row per group	DataFrame — original jitni hi rows, MultiIndex (group, original_index) ke saath
Example	<code>x['col'].sum()</code> , <code>x['col'].max()</code> , koi bhi single value	<code>x['new_col'] = ...; return x</code>
Isse milta-julta concept	<code>groupby().agg()</code> jaisa hi result	<code>groupby().transform()</code> jaisa (is lecture mein cover nahi hua, lekin concept yehi hai)

🗝️ **Sabse important line:** `apply()` "smart" hai — tum function ke andar jo bhi return karo, Pandas **usi ke hisaab se** decide karta hai final result Series banega ya DataFrame. Yehi cheez confusion create karti hai kyunki output ka "shape" tumhare function ke return

statement par depend karta hai — koi fixed rule nahi hai jaise `sum()` ya `mean()` mein hota hai.

PART D: Ab Lecture Ke Real Examples Isi Lens Se Dekho

Example 1 — `movies.groupby('Genre').apply(min)`

```
python

movies.groupby('Genre').apply(min)
```

🔑 `min` ek **built-in Python function** hai. Har group (mini-DataFrame) ke liye, `min()` us group ke har column ka minimum value nikal ke ek chhota result deta hai (jaise `.min()` method jaisa). Output: har genre ke liye har column ka min value — ek row per genre.

Modern pandas isko FutureWarning ke saath karta hai (`np.minimum.reduce` use karne ki salah deta hai) — bas warning hai, kaam sahi hota hai.

Example 2 — Custom scalar function: "A" se start hone wali movies count karna

```
python

def fun(x):
    return x['Series_Title'].str.startswith('A').sum()

a = movies[['Genre', 'Series_Title']].groupby('Genre').apply(fun)
```

🔑 Bilkul hamare toy example jaisa hi hai:

- `x` = ek genre ka poora mini-DataFrame (jaise Action ki 172 rows)
- `x['Series_Title'].str.startswith('A')` → boolean Series (True jaha title 'A' se start hoti hai)
- `.sum()` → True ko 1 maan ke total count
- Function scalar return kar raha hai → **result ek Series banega**, index = Genre names

```
Genre
Action      10
Adventure    2
Animation    2
...
```

Example 3 — Group-wise Rank add karna (transform-style)

```
python
```

```
def group_ranks(x):  
    x['group_rank_apply'] = x['IMDB_Rating'].rank(ascending=False, method='dense')  
    return x  
  
movies.groupby('Genre').apply(group_ranks)
```

🔑 Yaha function `poora (x)` (naye column ke saath) `return` kar raha hai — bilkul hamare `add_rank` toy example jaisa.

- `(x)` = ek genre ka poora mini-DataFrame
- `x['IMDB_Rating'].rank(...)` — is calculation mein **sirf isi group ki ratings** shaamil hain, poore dataset ki nahi (yehi groupby ka fayda hai — automatically "sirf apne group ke andar" scoped ho jaata hai)
- `return x` → poora modified DataFrame wapas
- Result: original jitni hi rows, bas naya `group_rank_apply` column add ho gaya, aur index mein ek extra level (Genre) aa gaya

Example 4 — Normalized rank (min-max scaling har group ke andar)

```
python  
  
def normalized_ranks(x):  
    a = x['IMDB_Rating'].min()  
    b = x['IMDB_Rating'].max()  
    x['normalized_rank'] = (x['IMDB_Rating'] - a) / (b - a)  
    return x  
  
movies.groupby('Genre').apply(normalized_ranks)
```

🔑 Isi pattern ka ek aur example — **transform-style apply**, jaha function multi-step logic (min nikaalo, max nikaalo, phir formula lagao) apne group ke andar hi karta hai, aur poora `(x)` wapas karta hai. Ye kaam `agg()` se **possible nahi hai**, kyunki `agg()` ek column ko ek value mein reduce karta hai — yaha humein **har row ke liye alag value chahiye** (based on us row ke group ki min/max), isliye `apply()` zaroori hai.

PART E: `apply()` `agg()` se Different Kyu/Kab Use Karna Hai

	<code>.agg()</code>	<code>.apply()</code>
Kaam	Fixed aggregation functions lagana (sum, mean, count...)	Koi bhi custom, multi-step, complex logic
Function ko kya milta hai	Ek COLUMN (Series) — agar dictionary form use karo	Poora GROUP ka DataFrame (<code>x</code>) — saare columns ek saath
Output size	Hamesha 1 row per group (reduce)	Depend karta hai — scalar ho toh 1 row/group, DataFrame ho toh original size
Speed	Tez (optimized/vectorized)	Thoda slow (Python-level loop internally chalta hai)
Kab use karo	Simple stats chahiye (total, average, count)	Row-level transform, multiple columns ek saath use karne hain, ya custom formula

🔑 **Simple rule of thumb:** Agar kaam sirf "is column ka sum/mean/max nikaalo" jaisa hai → `agg()` use karo (fast). Agar kaam "har row ko uske group ke context mein modify/calculate karo" jaisa hai (jaise rank, normalize, ya multi-column custom formula) → `apply()` use karo.

PART F: Common Confusions (FAQ style)

Q1: Function ke andar `x` exactly kya hai? A: `x` = us waqt jis group par kaam ho raha hai, uska **poora DataFrame** (sirf us group ke rows, baaki columns bhi shaamil — jab tak tumne pehle hi columns select na kiye ho).

Q2: `apply()` kitni baar function ko call karta hai? A: Utni baar jitne **unique groups** hain. Agar `Genre` mein 14 unique values hain, toh function 14 baar chalega — har baar ek alag group ka data lekar.

Q3: FutureWarning "operated on the grouping columns" ka matlab kya hai? A: Naye pandas versions mein, `x` ke andar grouping wala column (jaise `Genre`) by default include ho jaata hai, jo future mein exclude ho jayega. Warning se bachne ke liye `include_groups=False` pass kar sakte ho:

```
python
movies.groupby('Genre').apply(fun, include_groups=False)
```

Q4: Output mein do-level index (jaise `Action, 0`, `Action, 1`...) kyu aata hai? A: Jab function poora DataFrame return karta hai (transform-style), Pandas tumhare original

row-index ko as-is rakhta hai, aur **upar se ek naya level (group name) add kar deta hai** — isse "MultiIndex" kehte hain. `reset_index()` lagake isse normal columns mein convert kar sakte ho agar chahiye.

Q5: Kya `apply()` hamesha slow hota hai? A: Haan, `sum()`/`mean()` jaise built-in aggregate functions se `apply()` generally **slower** hota hai, kyunki pandas ke andar ek Python-level loop chalta hai har group ke liye. Jahan possible ho, `agg()` ya direct built-in methods use karna better performance deta hai — `apply()` tab use karo jab koi built-in option kaam na kare.

Quick Revision Cheatsheet — `apply()`

Concept	Yaad Rakhne Wali Baat
<code>apply(func)</code> kya karta hai	Har group ka poora DataFrame (<code>x</code>) function ko ek-ek karke deta hai
Function scalar return kare	Result = Series (1 value per group) — jaise mini-aggregation
Function DataFrame/same-shape return kare	Result = original size ka DataFrame with MultiIndex — jaise transform
<code>x</code> ke andar kya hota hai	Sirf USI group ke rows, poore DataFrame ke nahi
<code>agg()</code> vs <code>apply()</code>	agg = simple/fast aggregation, apply = flexible/custom/slower
<code>include_groups=False</code>	Grouping column ko (<code>x</code>) mein include hone se rokta hai (future-proof)
Speed	<code>apply()</code> generally slow — built-in methods available ho toh unhe prefer karo